SAPIENTIA
ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM

Sapientia Erdélyi Magyar Tudományegyetem

Marosvásárhelyi Kar

Számítástechnika szak

3D nyomtatási folyamat monitorizálás és hibadetektálás mesterséges intelligenciával

Készítette: Borbáth Mátyás-Levente

Témavezetők: Dr. Szántó Zoltán, Dr. Fehér Áron

2026

Kivonat

A 3D nyomtatási folyamat során különböző hibák léphetnek fel, amelyek anyagveszteséget, időveszteséget és akár a berendezés károsodását is okozhatják. A dolgozat célja egy olyan valós idejű monitorizáló rendszer fejlesztése, amely képfeldolgozási módszerekkel képes felismerni a nyomtatási hibákat. A javasolt megoldás egy többkamerás megfigyelőrendszerre épül, amely a nyomtatási folyamatot folyamatosan rögzíti, majd a képkockákat egy felhőalapú környezetben futó YOLOv8 neurális hálózat dolgozza fel. A modell egy saját annotált adathalmazon került betanításra, és 9 hibaosztály felismerésére képes, és 92.6% mAP50 pontossággal és 87.3% recall értékkel rendelkezik, illetve másodpercenként 3 képkockát dolgoz fel 23 ms inferencia idővel. A rendszer része egy értesítési mechanizmus is, amely anomália észlelése esetén azonnali visszajelzést ad a felhasználónak. A kísérleti eredmények alapján a rendszer alkalmas valós idejű hibadetektálásra, és hozzájárulhat a nyomtatási folyamat hatékonyságának növeléséhez, valamint az anyagveszteség csökkentéséhez.

Kulcsszavak: 3D nyomtatás,Konvoluciós Neuronháló, YOLOv8x, Rasberry Pi, Google Cloud, vertexAI, hiba, Terraform, Docker, Firebase

Tartalomjegyzék

1. Bevezető **1**

1.1. A projekt motivációja 1

1.1.1. Iparági relevancia és a nyomtatógazdálkodás kihívásai 2

1.1.2. A nyomtatási hibák műszaki és gazdasági hatása 3

1.1.3. Az automatizált monitorozás szükségessége 3

1.1.4. A projekt célkitűzései 4

2. Elméleti háttér **5**

2.1. 3D nyomtatásról 5

2.1.1. Nyersanyagok 6

2.1.2. Felmerülő hibák és limitációk 9

3. Mélytanulási modellek **18**

3.1. Piackutatás és a megfelelő AI modell kiválasztása 18

3.1.1. Lehetséges eszközök 18

3.2. A projekt megvalósításához kiválasztott modell – YOLOv8 19

3.2.1. Bevezetés 19

3.2.2. Főbb jellemzők 20

3.2.3. Modellvariánsok 20

3.2.4. Telepítés és alapszintű használat 21

3.2.5. Saját adathalmaz felhasználása tanításhoz 21

3.2.6. Annotációs formátum 22

4. Hardver- és szoftverkörnyezet **23**
4.1. A teljes rendszer . . . 23
4.1.1. Kamera Agent-Rasberry pi . . . 24
4.2. Szolgáltatások a PI-n . . . 24
4.2.1. MediaMTX . . . 24
4.2.2. Képkockák kinyerése . . . 25
4.2.3. Cloudflare alagút . . . 27
4.3. Cloud Stack . . . 27
4.3.1. Terraform keretrendszer . . . 28
4.3.2. Terraform működése . . . 28
4.3.3. Google Cloud Projekt - GCP . . . 29
4.3.4. Vertex AI végpont . . . 30
4.3.5. Service Accounts . . . 30
4.3.6. Pub/Sub Topics . . . 31
4.3.7. Google Cloud Függvények . . . 32
4.3.8. Firestore Collections . . . 33
4.3.9. Konténer képek . . . 34
4.4. Dashboard . . . 34
4.5. Github Action Workflows . . . 35
4.5.1. Terraform lépések automatizálása . . . 36
4.5.2. Docker konténer felépítésének automatizálása . . . 36

5. Adatgyűjtés és előfeldolgozás **39**
5.1. Cím 1 . . . 39

6. Modell betanítása és validálása **40**
6.1. Cím 1 . . . 40

7. Tesztelés és kiértékelés **41**
7.1. Összefoglalás . . . 41

8. Rendszerintegráció és alkalmazás **42**

9. Összefoglalás és jövőbeli irányok **43**

10. Mellékletek **44**

Irodalomjegyzék **44**

A. Függelék **48**

A.1. Alfejezet 48

A.1.1. Cím 48

Ábrák jegyzéke

1.1. 3D nyomtató farm 2

2.1. SLA nyomtató 6
2.2. FDM nyomtató 6
2.3. Döntési szempontok 6
2.4. Filament tekercsek 9
2.5. Szálazás a nyomtatás végén 10
2.6. Eltorzult darab 11
2.7. Elcsúszott rétegek 12
2.8. Teljes alul extrúdálás 13
2.9. Lokális alul extrúdálás 13
2.10. Felül extrúdált darab 14
2.11. Nyomtatás közben eldugult fúvóka és darab 15
2.12. Figyelmetlenségből a tárgyasztalon hagyott szerszám 16
2.13. Nem tapadó első réteg 16
2.14. Hibás nyomtatást követő spagetti halmaz 17

4.1. A rendszer 24
4.2. MediaMTX stream pipeline 25
4.3. Terraform Lépések[1] 29
4.4. A `judge` konténer image életciklusa 34
4.5. A dashboard normál működési állapotban 35
4.6. A dashboard anomália detektálás esetén 35

4.7. A CI/CD pipeline lépései 37

1. fejezet

Bevezető

1.1. A projekt motivációja

Az additív gyártás – ezen belül is az FDM (Fused Deposition Modelling) alapú 3D nyomtatás – az elmúlt évtizedben kiemelkedő szerepet töltött be a modern gyártóiparban. A technológia lehetővé teszi egyedi geometriák, kis sorozatú alkatrészek és funkcionális prototípusok gyors, gazdaságos előállítását, melyre a hagyományos szubsztraktív eljárások nem képesek ilyen rugalmassággal. Ezen előnyök eredményeképpen a 3D nyomtatás napjainkra az ipari termelésen túl az egészségügytől az űrtechnológiáig számos területen meghonosodott.

Ugyanakkor a technológia széleskörű elterjedésével párhuzamosan az üzemeltetési kihívások is egyre összetettebbekké válnak. A nyomtatási folyamatok – különösen FDM technológia esetén – számos tipikus meghibásodásra hajlamosak, amelyek anyagveszteséget, időveszteséget, illetve súlyosabb esetben a berendezés megrongálódását okozhatják. Ilyen körülmények között a hibák kézi, szemrevételezésen alapuló felismerése nem csupán lassú és körülményes, hanem nagyüzemi környezetben gyakorlatilag kivitelezhetetlen.

Jelen fejezet ismerteti azokat az iparági és műszaki tényezőket, amelyek indokolttá teszik egy valós idejű, mesterséges intelligencián alapuló hibadetektor rendszer kifejlesztését, majd összefoglalja a projekt konkrét célkitűzéseit.

1.1.1. Iparági relevancia és a nyomtatógazdálkodás kihívásai

A 3D nyomtatás iparági térhódítása az elmúlt évtizedben látványos volt. A gyártóüzemekben egyre elterjedtebbé válnak az úgynevezett nyomtatófarmok, ahol egyetlen helyszínen tíz vagy akár száz nyomtatóegység üzemel egyidejűleg [1]. Ilyen léptékű infrastruktúra esetén a manuális felügyelet – azaz a kezelő személyes jelenlétével megvalósuló ellenőrzés – nem valósítható meg hatékonyan: a karbantartó személyzet sem az egyes gépek állapotát, sem az éppen zajló nyomtatásokat nem képes folyamatosan nyomon követni.

1.1. ábra. 3D nyomtató farm [1]

A farm-üzemeltetés sajátos kihívást jelent a hibadetektálás szempontjából is. Ha egyetlen gépen bekövetkező hiba – például szálazás, réteg-eltolódás vagy tapadásvesztés – észrevétlen marad, az anyag- és időveszteség sokszorozódhat: a nyomtató akár órákon át termel selejtet, mire a kezelő beavatkozik. Súlyosabb esetben a fúvóka károsodása vagy a tárgyasztal megsérülése komoly javítási

költséget von maga után. Az automatizált monitorozás ebben a kontextusban nem csupán kényelmi eszköz, hanem gazdasági szükségszerűség [2].

1.1.2. A nyomtatási hibák műszaki és gazdasági hatása

A 3D nyomtatási folyamat több okból is hibásodhat meg: a nem megfelelően kondicionált filamenttől és hibás szeletelési beállításoktól egészen a kalibrációs eltérésekig és hardveres meghibásodásokig. A leggyakoribb és legsúlyosabb következményekkel járó hibatípusok a következők:

- **Tapadásvesztés és spagetti:** Az első réteg elválása a tárgyasztaltól az egyik legelterjedtebb és legnehezebben megelőzhető hiba. Ha a darab elveszíti rögzítettségét, a fúvóka a levegőbe extrudálja az anyagot, egy jellegzetes, rendezetlenül összegubancolódott szálkupacot – köznyelvben „spagettit" – hozva létre. Az anyagveszteség ilyenkor akár több tíz gramm is lehet.
- **Réteg-eltolódás (layer shift):** A koordináta-rendszerbeli eltérés miatt a nyomtatott rétegek elcsúsznak egymáshoz képest, a darab szerkezete meggyengül vagy teljesen használhatatlanná válik.
- **Alul- és felülextrúdálás:** Az anyagadagolás egyensúlytalansága a darab struktúrális integritását veszélyezteti; szélső esetben fúvókaduguláshoz vezet.
- **Szálazás (stringing):** Bár nem feltétlenül teszi funkcióképtelenné a darabot, az esztétikai és precíziós igényekre érzékeny alkalmazásokban elfogadhatatlan minőséget eredményez.

Ezek a hibák nemcsak az egyes darabok minőségét rontják le, hanem – ha felügyelet nélkül maradnak – a gyártóeszköz élettartamát is csökkenthetik. A nyomtatófarmok üzemeltetői számára különösen fontos a korai, automatikus jelzés, hiszen a kiesési idő és az anyagköltség közvetlen üzleti veszteséget jelent.

1.1.3. Az automatizált monitorozás szükségessége

Az automatizált képfeldolgozás és a mélytanulási módszerek rohamos fejlődése napjainkra megteremtette az alapot a valós idejű ipari hibadetekcióhoz. A konvolúciós neurális hálózatok (CNN-ek)

képesek vizuális mintázatok felismerésére akkor is, ha azok az emberi szem számára nehezen, vagy csak figyelmes vizsgálattal azonosíthatók. Az Obico (korábban The Spaghetti Detective) projekt például egy nyílt forráskódú, mesterséges intelligencián alapuló megoldást nyújt az FDM nyomtatók felügyeletéhez, és igazolta, hogy ilyen rendszerek a valóságban is megbízhatóan működtethetők [2].

Ugyanakkor az elérhető megoldások jellemzően egyetlen szögből figyelik a nyomtatót, ami korlátozza az észlelhető hibatípusok körét. Egy, a tárgyasztalt több kameraállásból felügyelő rendszer lényegesen pontosabb képet ad a folyamatról, és olyan hibák is detektálhatóvá válnak, amelyek egyetlen nézőpontból nem látszanak.

1.1.4. A projekt célkitűzései

A fentiekben ismertetett iparági igények és műszaki lehetőségek alapján a jelen projekt célja egy olyan skálázható, valós idejű monitorozó- és hibadetektor rendszer megtervezése és megvalósítása, amely:

- Minden felügyelt nyomtatót egyszerre, **több kameraállásból** figyel;
- **Konvolúciós neurális hálózat** segítségével automatikusan azonosítja a leggyakoribb FDM hibatípusokat;
- A detektált hibát **kategorizálja** – megkülönböztetve a nyomtatás közbeni beavatkozást igénylő, illetve az azonnali leállítást indokló eseteket;
- A felhasználót **valós időben értesíti** a szükséges teendőkről;
- **Skálázható** architektúrán alapul, hogy nyomtatófarmok esetén is alkalmazható legyen.

A rendszer megvalósításával jelentős anyag-, idő- és költségmegtakarítás érhető el, mivel a hibák korai felismerése megakadályozza a selejtes darabok keletkezésének elhúzódását és a berendezések esetleges sérülését.

2. fejezet

Elméleti háttér

2.1. 3D nyomtatásról

Az additív gyártás olyan gyártási folyamatok összefoglaló neve, amelyek során az elkészítendő darabot rétegről rétegre építjük fel. Ez alapvetően szemben áll a szubsztraktív gyártással, amelynek során az alkatrészt egy szolid alapanyagból, anyageltávolítással – például marással vagy esztergálással – állítják elő.

A 3D nyomtatás az additív gyártás egyik legismertebb megvalósítása: egy digitális CAD-modellből kiindulva, számítógép vezérlésével, rétegről rétegre építi fel a kívánt tárgyat. A folyamat első lépése a digitális modell előkészítése: egy ún. szeletelőprogram a háromdimenziós geometriát egymással párhuzamos, vékony szeletekre bontja, majd ezekből G-kódot generál – ez az utasítássorozat irányítja a nyomtató mozgását és anyagadagolását. A nyomtató a G-kód alapján, rétegenként rakja fel az anyagot, amíg a darab el nem készül.

A piacon számos 3D nyomtatási technológia érhető el, amelyek eltérő alapanyagokat és működési elveket alkalmaznak. A legelterjedtebb az **FDM** (Fused Deposition Modelling), amelynek során egy hőre lágyuló műanyag filamentet olvasztanak meg és extrudálnak vékony szálként a tárgyasztalra. Az **SLA** (Stereolithography) nyomtatók folyékony fotopolimer gyantát szilárdítanak meg precízen irányított lézersugárral; rendkívül finom részletek nyomtatására alkalmasak, ugyanakkor az elkészült darabok jelentős utómunkát igényelnek. Emellett megemlítendők a **PolyJet**, **Binder Jetting**, valamint a fémnyomtatásban alkalmazott **DMLS/SLM** technológiák is. Jelen dolgozat az FDM nyomtatók üze-

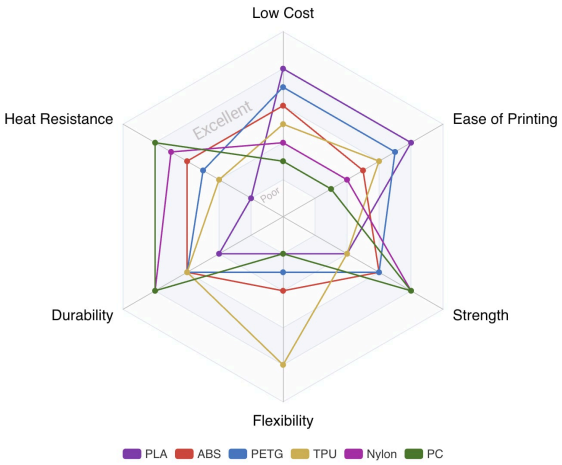
meltetése során felmerülő hibákkal foglalkozik, ezért a továbbiakban kizárólag ezzel a technológiával foglalkozunk.

2.1. ábra. SLA nyomtató[1]

2.2. ábra. FDM nyomtató[2]

2.1.1. Nyersanyagok

Az FDM nyomtatók nyersanyaga a filament. A felhasználási igényektől függően számos anyagfajta áll rendelkezésre, mindegyik eltérő mechanikai, hőtani és kémiai tulajdonságokkal rendelkezik [3].



2.3. ábra. Döntési szempontok[3]

[1] *Fotó*:https://ro.pinterest.com/pin/338895940694784238/

[2] *Fotó*:https://eu.store.bambulab.com/products/a1?id=599117150694776844

PLA – Politejsav

A PLA (Politejsav, angolul: *Polylactic Acid*) a legelterjedtebb filament a hobbi- és félipari 3D nyomtatás területén. Megújuló alapanyagokból, leggyakrabban kukoricakeményítőből, cukornádból vagy tápióka gyökérből állítják elő [3]. Fontos megjegyezni, hogy biológiai lebomlása kizárólag ipari körülmények között valósítható meg, ahol megfelelő hő, páratartalom és mikrobiológiai aktivitás biztosított; e feltételek hiányában a PLA ugyanolyan környezeti terhelést jelent, mint bármely más műanyag. Olvadáspontja alacsonyabb más filamentekhez képest, mechanikai szilárdsága közepes kategóriába sorolható, rugalmassága alacsony. Hátrányai közé tartozik a magas hőmérséklettel és páratartalommal, valamint az UV-sugárzással szembeni érzékenysége.

ABS – Akrilnitril-butadién-sztirol

Az ABS (Akrilnitril-butadién-sztirol, angolul: *Acrylonitrile Butadiene Styrene*) a hobbi felhasználók körében kevésbé elterjedt, mivel nyomtatása során mérgező gázok keletkezhetnek; megfelelő szellőztetés mellett azonban ez a kockázat teljes mértékben kezelhető [3]. Az iparban előszeretettel alkalmazzák, mivel mechanikai tulajdonságai összességében felülmúlják a PLA-ét. Nyomtatása igényesebb, mivel az anyag erősen hajlamos a torzulásra, amennyiben a nyomtatótérben hőingadozás lép fel. Mechanikai keménysége magas, rugalmassága jobb a PLA-énál. Az ABS-ből készült alkatrészek időtállóak, ellenállnak a nedvességnek és az UV-sugárzásnak egyaránt.

PETG – Glikolizált polietilén-tereftalát

A PETG (Glikolizált polietilén-tereftalát, angolul: *Polyethylene Terephthalate Glycol*) a felhasználóbarátság, a mechanikai szilárdság és a rugalmasság kedvező egyensúlyát kínálja [3]. Az anyag alapja a műanyag palackok gyártásánál is alkalmazott PET, amelyet glikollal módosítanak; ennek eredményeképpen az anyag kevésbé törékennyé és minőségileg stabilabbá válik. Mechanikai szilárdsága magas, rugalmassága közepes. Rendkívül ellenálló a nedvességgel szemben, és kevésbé hajlamos a torzulásra más filamentekhez képest. Tárolása és szárítása gondosságot igényel.

[3] *Fotó*:https://all3dp.com/1/3d-printer-filament-types-3d-printing-3d-filament/

TPU – Termoplasztikus poliuretán

A TPU (Termoplasztikus poliuretán, angolul: *Thermoplastic Polyurethane*) egy rendkívül rugalmas, gumiszerű filament, amely ruganyosságáról, keménységéről és ütésálló képességéről ismert [3]. A merev filamentekkel ellentétben hajlítható, nyújtható és összenyomható, miközben képes az ütések elnyelésére. Mechanikai keménysége a merev anyagokhoz képest alacsonyabb, rugalmassága azonban kiemelkedően magas. Folyamatos igénybevételnek kitett alkatrészek – például szíjak és tömítések – nyomtatására különösen alkalmas. Ruganyos tulajdonsága miatt az anyag hajlamos a szálazásra, ezért nyomtatási sebessége alacsonyabb a szilárd anyagokénál, és az adagoló rendszer kalibrálása fokozott figyelmet igényel.

Nylon – Poliamid

A Nylon (Poliamid, angolul: *Polyamide*) elsősorban ipari alkalmazásokban elterjedt filament, kiemelkedő szilárdsága és tartóssága miatt [3]. Nyomtathatósága a fejlettebb fúvókatechnológiák és a zárt nyomtatóterek elterjedésével vált széles körben elérhetővé, amelyek jobb kontrollt biztosítanak a nedvességtartalom és a hőmérséklet felett. Mechanikai szilárdsága elsőrendű, rugalmassága kiváló, tartóssága felülmúlja a legtöbb más filamentét. Erősen higroszkópikus anyag, ezért tárolása és felhasználás előtti szárítása elengedhetetlen.

PC – Polikarbonát

A polikarbonát (angolul: *Polycarbonate*) a rendelkezésre álló filamentek közül kiemelkedő mechanikai szilárdságáról és tartósságáról ismert, emellett rendkívül ellenálló mind a hővel, mind a nedvességgel szemben [3]. Nyomtatása jelentős technikai követelményeket támaszt: a zökkenőmentes feldolgozáshoz a fúvóka hőmérsékletének a 260–310 °C tartományban kell lennie, a tárgyasztalt pedig zárt nyomtatótérbe kell elhelyezni a hőingadozás megakadályozása érdekében.

Egyéb nyersanyagok

A fentieken túl számos egyéb filamenttípus is elérhető, amelyek jellemzően az ismertetett alapanyagok keverékei vagy módosított változatai [3]. Ilyen például a selyem PLA, amelynek mechanikai

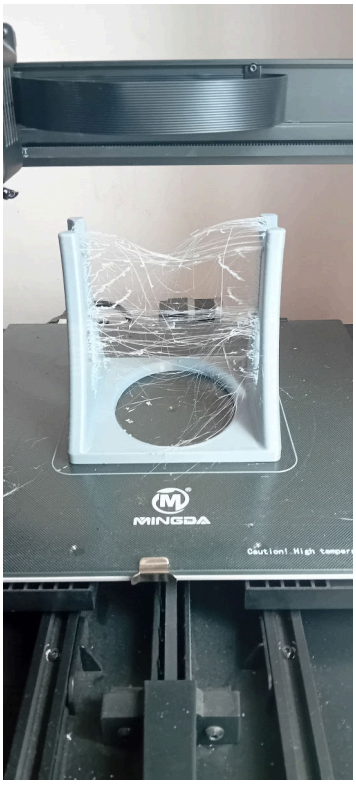
szilárdsága alacsonyabb, azonban fényesebb, selymes textúrájú felületet eredményez, így esztétikai célú nyomtatásra – például dísztárgyak készítésére – különösen alkalmas. A fa-PLA fatörmelékkel kevert PLA-variáns, amely nyomtatás után fás tapintású felületet kölcsönöz a darabnak; hasonló elven készülnek a fémrészecskékkel kevert fém-PLA kompozitok is.

2.4. ábra. Filament tekercsek

2.1.2. Felmerülő hibák és limitációk

A 3D nyomtatási folyamat során számos hiba léphet fel, amelyek eredete a nedves alapanyagtól a helytelen kalibrációig, illetve komolyabb hardveres meghibásodásig terjedhet [4].

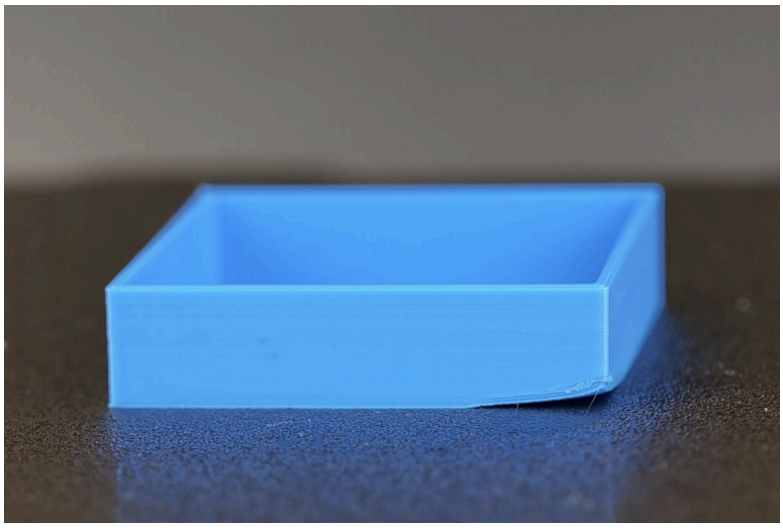
Stringing – Szálazás



2.5. ábra. Szálazás a nyomtatás végén

A szálazás fő jellegzetessége apró, szinte áttetsző szálak megjelenése a darab elemei között. A jelenség oka, hogy a fúvóka mozgás közben olvadt anyagot húz maga után az egyik pozícióból a másikba való áthaladás során. Leggyakoribb kiváltó oka a nem megfelelően kiszárított filament használata [5]. Az így keletkező szálak utómunkával eltávolíthatóak, azonban nem garantált, hogy nyomuk teljes mértékben megszüntethető. Emellett helytelen visszahúzási érték, túl magas fúvóka-hőmérséklet vagy szennyezett fúvóka is okozhatja a hibát.

Warping – Vetemedés

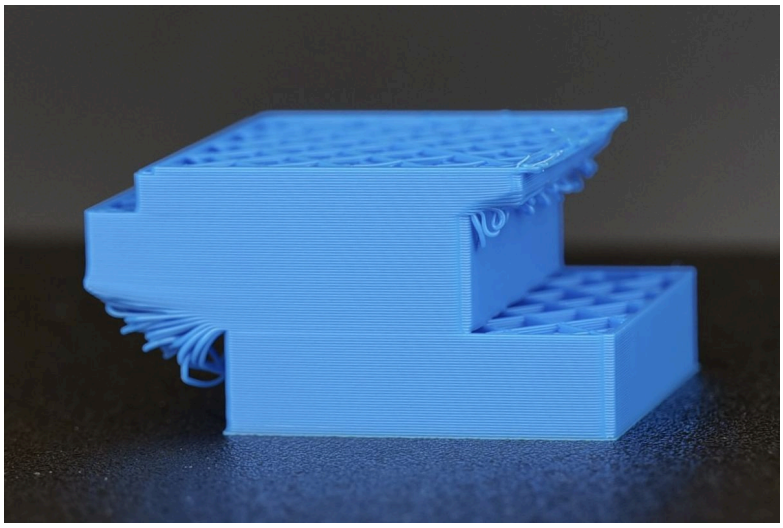


2.6. ábra. Vetemedett darab[4]

Az eltorzulás legtöbbször olyan darabok esetén jelentkezik, amelyek alján éles szögek találhatóak, például négyszög vagy más sokszög alapú geometriáknál [6]. A tárgyasztal hőmérséklete és tisztasága szintén meghatározó tényező a megfelelő tapadás fenntartásában. A hiba megelőzhető a szeletelőprogramban alkalmazott ún. *brim* (perem) elemmel, amely bónusz tapadási felületet biztosít a sarokpontoknál.

[4] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/printed-model-warping

Layer shift – Réteg-eltolódás



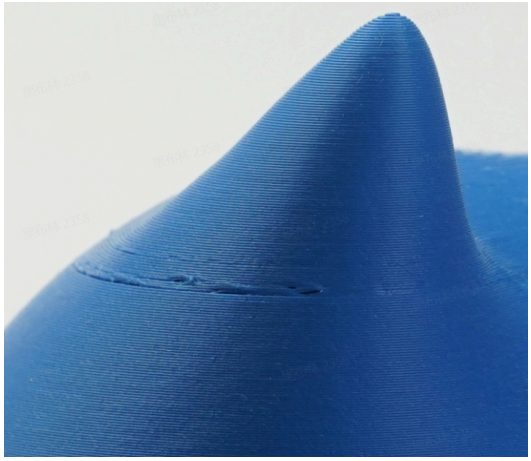
2.7. ábra. Elcsúszott rétegek[5]

A réteg-eltolódás a nyomtatott darab számára végzetes hiba [7]. Jellemzően akkor lép fel, amikor eltérés keletkezik a G-kódban tárolt koordináta és a fúvóka tényleges térbeli koordinátái között. Három fő kiváltó oka azonosítható: túl nagy nyomtatási sebesség; helytelen Z-offset érték, amely miatt a fúvóka beleakadhat a darabba és koordinátakülönbséget idézhet elő; valamint a nyomtatót ért külső fizikai behatás, illetve laza mozgósínek vagy merevítők.

[5] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/layer-shifts

Under extrusion – Alul-extrúdálás

2.8. ábra. Teljes alul extrúdálás[6]



2.9. ábra. Lokális alul extrúdálás[7]

Az alul-extrúdálásnak két fajtája különböztethető meg: a teljes és a lokális alul-extrúdálás [8]. A hiba eredete lehet hardveres és szoftveres egyaránt. Hardveres okok közé tartozik a PTFE-cső eldugulása vagy idegen test bejutása, a filamenttekercs beakadása miatti anyaghiány, fogaskerék-probléma, illetve részleges dugulás a fúvókában vagy a hotendben. Szoftveres oldalon a helytelen *Maximum Volumetric Speed* vagy *Flow Ratio* beállítás válthatja ki. A korszerű nyomtatók automatikus adagolókalibráló rendszerrel rendelkeznek, amely segít megelőzni e hibát.

[6] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/under-extrusion
[7] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/under-extrusion

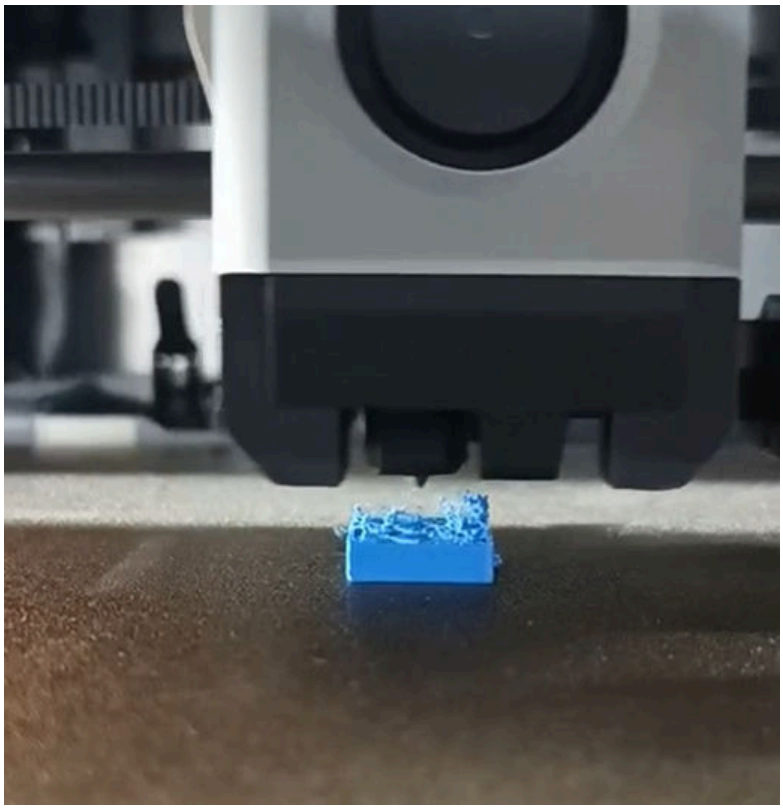
Over extrusion – Felül-extrúdálás

2.10. ábra. Felül extrúdált darab[8]

A felül-extrúdálás az alul-extrúdálással ellentétes jelenség: ebben az esetben a fúvókához a szükségesnél több anyag jut, ami a darab felszínének minőségét rontja és méretbeli pontatlanságot okoz.

[8] *Fotó*:https://www.3dsourced.com/guides/3d-print-over-extrusion/

Nozzle Clog – Fúvóka-dugulás



2.11. ábra. Nyomtatás közben eldugult fúvóka és darab[9]

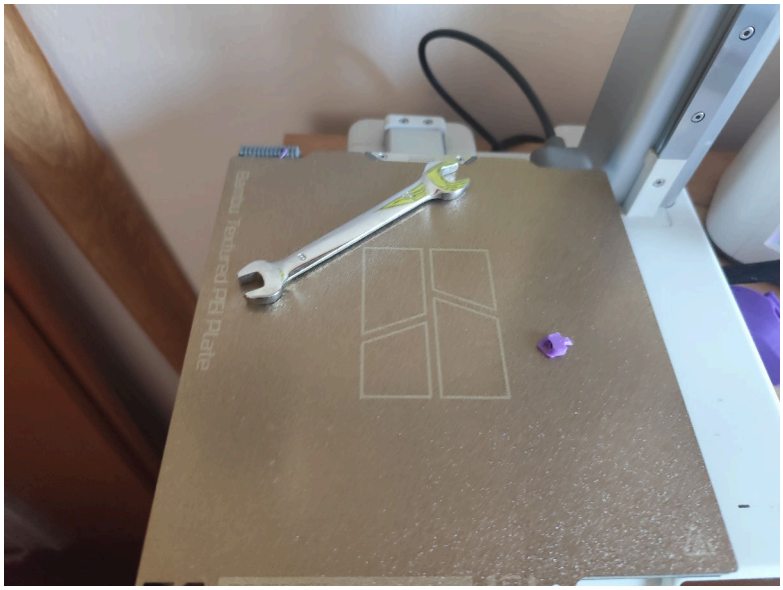
A fúvóka-dugulás az egyik legsúlyosabb következményekkel járó üzemzavar [9]. Leggyakoribb oka, hogy a PTFE-csövön és a forró zónán áthaladó filament por vagy egyéb szennyeződés nyomait tartalmazza, amelyek lerakódnak és dugulást okoznak a fúvókában. Amennyiben a hiba nem kerül időben felismerésre, az olvadt anyag felgyűlhet a fúvókában, és belülről mechanikai sérülést okozhat, ezzel jelentős anyagi kárt idézve elő.

Foreign object on the printbed – Idegen tárgy a tárgyasztalon

Ez a hibatípus nem hardveres és nem szoftveres eredetű, hanem kizárólag a kezelő figyelmetlenségéből ered [4]. Leggyakoribb eset, hogy az előkészítést követően a kezelő elmulasztja ellenőrizni a

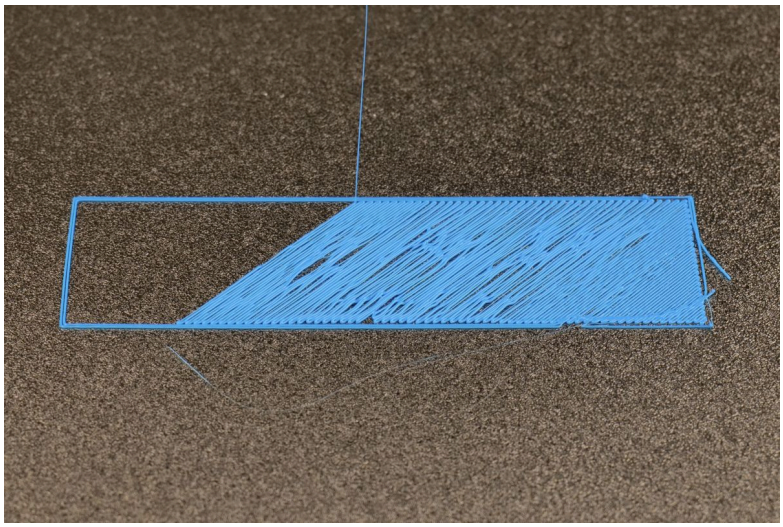
[9] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/clog

tárgyasztal állapotát, vagy karbantartás után szerszámot illetve egyéb tárgyat hagy rajta. Az idegen tárgyak súlyos mechanikai kárt okozhatnak a fúvókában és a nyomtatófejben, mivel a legtöbb nyomtató nem rendelkezik a tárgyasztal szabad felületének automatikus érzékelésére alkalmas rendszerrel.



2.12. ábra. Figyelmetlenségből a tárgyasztalon hagyott szerszám

First layer not sticking – Az első réteg nem tapad



2.13. ábra. Nem tapadó első réteg[10]

Az első réteg megfelelő letapadása a nyomtatási folyamat legkritikusabb mozzanata [10]. Amennyiben az első réteg nem tapad a tárgyasztalhoz, a darab elvész és jelentős anyagveszteséggel jár. Főbb kiváltó okai: nem megfelelő tisztaságú tárgyasztal, helytelen fúvóka–tárgyasztal távolság (Z-offset), valamint pontatlan asztalszintezés. Megfelelő beállításokkal és a tárgyasztal rendszeres karbantartásával a hiba könnyen megelőzhető.

Spagetti – Az anyag a levegőbe adagolódik

A spagetti-hiba leggyakoribb előidézője az első réteg sikertelen tapadása [11]. Ebben az esetben a nyomtató a G-kódban rögzített utasításokat változtatás nélkül hajtja végre, és a fúvóka a tapadás hiánya ellenére folytatja az anyag adagolását, mindaddig, amíg a nyomtatás befejeződik vagy a kezelő manuálisan leállítja azt. Fontos kiemelni, hogy a tapadás elvesztése a nyomtatási folyamat bármely szakaszában bekövetkezhet, különösen helytelen tartószerkezet-beállítások alkalmazása, vagy a supportok tárgyasztaltól való elválása esetén.

2.14. ábra. Hibás nyomtatást követő spagetti halmaz

[10] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/first-layer-not-sticking

3. fejezet

Mélytanulási modellek

3.1. Piackutatás és a megfelelő AI modell kiválasztása

3.1.1. Lehetséges eszközök

Képfelismerési feladatok megvalósítására számos gépi tanulási keretrendszer áll rendelkezésre, így a projekt megvalósítása során nem volt szükség egy teljes neurális háló felépítésére a semmiből. A megfelelő eszköz kiválasztásánál több szempontot is figyelembe kellett venni: a modell pontosságát, valós idejű felhasználhatóságát, integrálhatóságát, valamint a rendelkezésre álló hardver- és szoftverkörnyezettel való kompatibilitást. Kiemelt szempont volt, hogy a betanított modell könnyen illeszthető legyen egy Jupyter Notebook alapú, majd Google Cloud infrastruktúrára épülő pipeline-ba, ezzel csökkentve a tanítás és futtatás során felmerülő hardveres korlátokat.

Faster R-CNN

A Faster R-CNN (*Region-based Convolutional Neural Network*) egy kétfázisú objektumdetekciós neurális hálózat, amelyet 2015-ben a Microsoft Research kutatói publikáltak [12]. Működése három egymást követő lépésre bontható. Az első fázisban egy előre betanított konvolúciós hálózat vizuális jellemzőtérképet (*feature map*) készít a bemeneti képből. Ezt követi a *Region Proposal Network* (RPN), amely a jellemzőtérkép alapján jelöltterületeket generál, és lényegesen gyorsabb a korábbi, manuális régiókereső algoritmusoknál. A harmadik lépésben az RoI (*Region of Interest*) pooling egy-

séges méretre skálázza a javasolt régiókat, majd egy másodlagos osztályozó hálózat minden régiót külön kiértékel, meghatározza az osztálycímkéket, és finomítja a határoló dobozok koordinátáit.

A Faster R-CNN előnye a kétfázisú feldolgozásból fakadó magas detekciós pontosság, és az előre betanított konvolúciós gerinc könnyen cserélhető más hálózatokra. Hátrányaként az alacsony inferenciasebesség emelhető ki, ami kizárja a valós idejű alkalmazásokban való felhasználást. Emellett a modell mérete és erőforrásigénye lényegesen nagyobb, mint az egyfázisú detektoroké, és a Jupyter Notebook-alapú pipeline-ba való integrálása is jóval összetettebb.

YOLOv5

A YOLOv5 (*You Only Look Once*, 5. verzió) egy egyfázisú objektumdetekciós neurális hálózat, amelyet az Ultralytics cég adott ki 2020-ban [13]. Az iparban széleskörűen elterjedt, mivel könnyen betanítható, jól dokumentált, és minimális konfigurációval felhasználható. Az architektúra három fő részből áll: a *gerinc* (*backbone*) a Cross Stage Partial Darknet segítségével a képből kinyeri a vizuális jellemzőket; a *nyak* (*neck*) összevonja a különböző méretarányú jellemzőket, biztosítva a többméretű objektumdetekciót; a *fej* (*head*) horgonydobozok (*anchor boxes*) alapján három különböző felbontásban állítja elő a detekciós kimenetet.

A YOLOv5 alacsony inferenciaideje miatt alkalmas valós idejű felismerésre, a hiperparaméterek és augmentációk előre konfigurálva vannak, és a veszteségfüggvény is jól hangolt. Bár a projekt szempontjából megfelelő jelölt lett volna, az Ultralytics által 2023-ban kiadott YOLOv8 mind architektúra, mind teljesítmény tekintetében felülmúlja elődjét, ezért a választás arra esett.

3.2. A projekt megvalósításához kiválasztott modell – YOLOv8

3.2.1. Bevezetés

A YOLOv8 (*You Only Look Once*, 8. verzió) az Ultralytics cég által 2023-ban kiadott objektumdetekciós modellcsalád [14]. A valós idejű objektumdetektálásban nyújtott kiemelkedő teljesítménye, valamint a könnyű integrálhatósága indokolta a választást, mivel a 3D nyomtatási folyamat élő kamerabemenetről történő felügyeletéhez és a hibák automatikus azonosításához mindkét tulajdonság

elengedhetetlen.

3.2.2. Főbb jellemzők

A YOLOv8 keretrendszer esetében az Ultralytics teljes mértékben újragondolta a modell architektúráját. Az előző verzióhoz képest egy saját fejlesztésű keretrendszerre tért át, amelynek köszönhetően a modell gyorsabbá és stabilabbá vált, ugyanakkor ez visszafelé inkompatibilitást jelent a YOLOv5-tel szemben [14]. A keretrendszer az objektumdetekción kívül szegmentálást, pozícióbecslést, osztályozást és objektumkövetést is támogat. A tanítási és detekciós folyamat segédszkriptek nélkül, API-hívásokon vagy parancssori felületen (CLI) keresztül is vezérelhető. A betanított modell exportálható többek között ONNX, CoreML és TensorRT formátumokba is.

3.2.3. Modellvariánsok

A YOLOv8 több méretű változatban érhető el, amelyek különböző felhasználási célokhoz és hardverkörnyezethez igazíthatók [14].

3.1. táblázat. YOLOv8 modellvariánsok

Modell	**Méret**	**Leírás**
`yolov8n`	Nano	A legkisebb variáns, rendkívül gyors és minimális hardverigényű.
`yolov8s`	Small	Kis méretű modell, gyors inferenciával, alacsony erőforrásigénnyel.
`yolov8m`	Medium	Kiegyensúlyozott kompromisszum sebesség és pontosság között.
`yolov8l`	Large	Nagy pontosságú modell, erősebb GPU-t igényel.
`yolov8x`	Extra Large	A legnagyobb pontosságot nyújtó variáns, egyúttal a legtöbb erőforrást igénylő.

3.2.4. Telepítés és alapszintű használat

A csomag a Python csomagkezelőn keresztül telepíthető [14]:

```
pip install ultralytics
```

Egyszerű képfeldolgozási példa:

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
results = model("stringing.jpg")
results.show()
```

3.2.5. Saját adathalmaz felhasználása tanításhoz

A YOLOv8 modell saját adathalmazon is betanítható, amennyiben a képek megfelelően fel vannak annotálva és az exportált annotációk formátuma YOLO-kompatibilis [14]. Ehhez egy `data.yaml` konfigurációs fájlra van szükség, amely megadja a tanítási és validációs adathalmazok elérési útjait, az osztályok számát és elnevezéseit.

A `data.yaml` konfigurációs fájl szerkezete:

```
train: images/train
val: images/val
nc: 8
names: ['stringing', 'warping', 'layer_shift', 'under_extrusion',
        'over_extrusion', 'nozzle_clog', 'foreign_object_on_print_area',
        'not_sticking', 'spagetti']
```

Tanítási folyamat indítása CLI-n keresztül:

```
yolo task=detect mode=train model=yolov8n.pt data=data.yaml epochs=50 imgsz=640
```

3.2.6. Annotációs formátum

YOLO-formátumban minden képfájl mellé egy azonos nevű `.txt` fájl tartozik. Minden sor egy objektumot ír le; egyetlen fájl több objektum leírását is tartalmazhatja:

```
<class_id> <x_center> <y_center> <width> <height>
```

Az értékek lebegőpontos számok, a kép méretéhez arányosan 0 és 1 közé normalizálva. Az annotáláshoz a CVAT (*Computer Vision Annotation Tool*) szoftvert [15] alkalmaztuk.

3.2. táblázat. YOLOv8 előnyei és hátrányai a projekt szempontjából

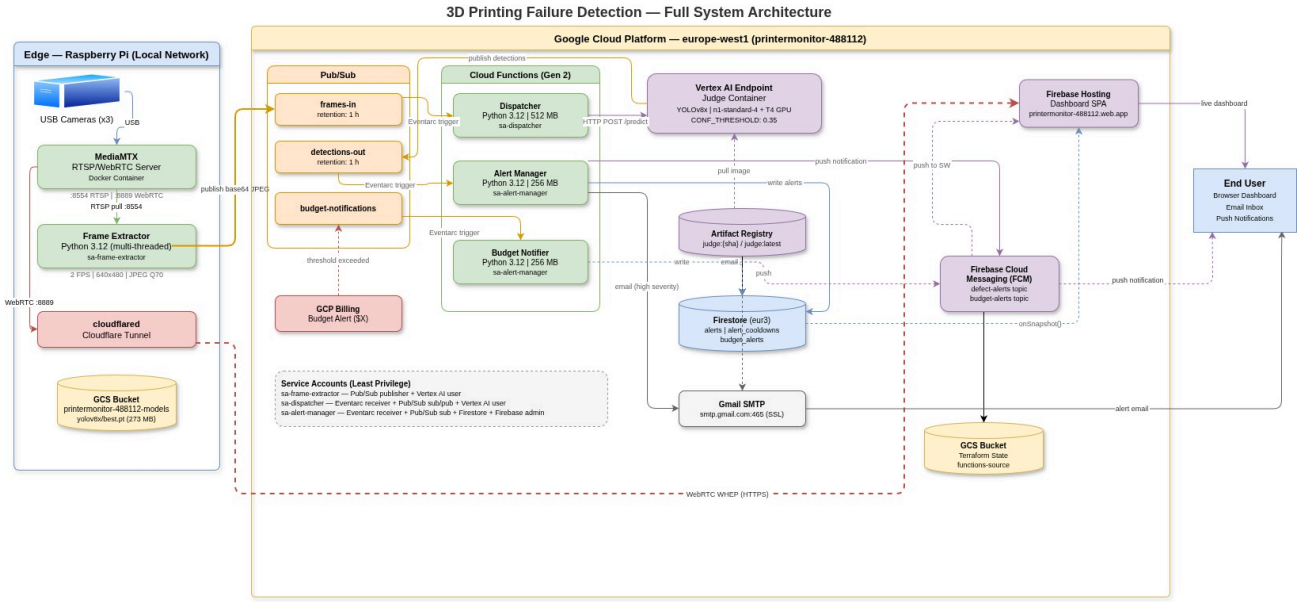
Előnyök	**Hátrányok**
Valós idejű objektumdetektálás	Nagy modellek (pl. YOLOv8x) jelentős GPU-erőforrást igényelnek
Egyszerű és megismételhető tanítási folyamat	Kis adathalmazon hajlamos a túltanulásra
Kis modellek is magas pontosságot érnek el	Zsúfolt vagy részben takart objektumok detektálása nehézkes
Docker-konténerizációs támogatás	Valós idejű feldolgozás teljesítménye hálózati késleltetéstől függhet
Aktív fejlesztői közösség és folyamatos frissítések	Egyedi osztályokhoz saját adathalmaz gyűjtése szükséges

4. fejezet

Hardver- és szoftverkörnyezet

4.1. A teljes rendszer

A rendszer egy edge–cloud hibrid architektúrára épül, amelynek két fő pillére van: egy helyszínen elhelyezett Raspberry Pi edge eszköz, valamint a Google Cloud Platform felhőinfrastruktúrája. A Pi feladata a kamerás képrögzítés és az adatok előfeldolgozása, míg a felhő végzi a gépi tanulás alapú hibadetekciót, az adattárolást és a felhasználói értesítéseket. Az adatfolyam iránya: Raspberry Pi → Pub/Sub (`frames-in`) → Dispatcher Cloud Function → Vertex AI végpont (YOLO inferencia) → Pub/Sub (`detections-out`) → Alert-Manager Cloud Function → Firestore + e-mail értesítés → Dashboard.



4.1. ábra. A rendszer

4.1.1. Kamera Agent-Rasberry pi

A kamera képek előállításért felelő kamera Agent szerepét egy rasberry pi 4b 8gb tölti be,[16] 3 darab rácsatolt webkamerával kiegészítve. A rasberry pi mint edge eszköz egy elég verszatilis fegyver a kezünkben. Keveset is fogyaszt, megfelelő számú usb porttal is rendelkezik, és az erőforrások is megfelelőek számunkra. A rasberry Os, pedig tökéletes a szükséges docker konténerek és python scriptek futtatására.

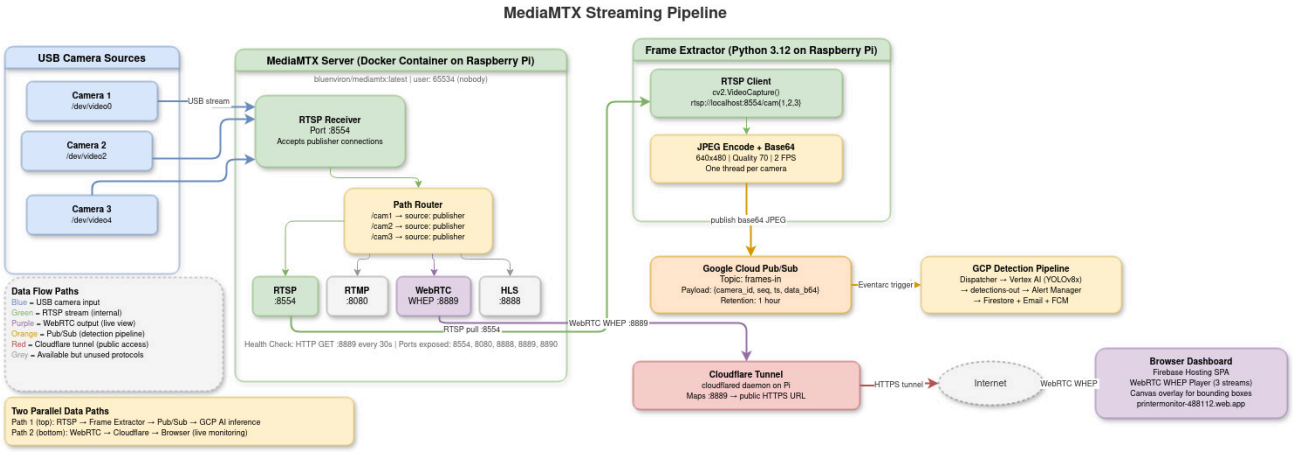
4.2. Szolgáltatások a PI-n

A pi-n három szolgáltatás fut, melyek az indítás után azonnal elindulnak systemd szolgáltatásként. Ezek nélkül a rendszer nem valósulhatna meg.

4.2.1. MediaMTX

A MediaMTX (korábban rtsp-simple-server) egy könnyűsúlyú, nyílt forráskódú médiaközvetítő szerver, mely Go programozási nyelven íródott[17]. Szerepe a rendszerben, hogy a webkamerák által

előállított nyers videófolyamokat RTSP protokoll szerint tegye elérhetővé a hálózaton. A pi-n systemd szolgáltatásként fut, így az eszköz indításakor automatikusan elindul, emberi beavatkozás nélkül. Konfigurációs fájlja definiálja a három kamerastream elérési útját, a kódolási paramétereket, valamint az újracsatlakozási logikát. A H264-es kódolású RTSP végpontokat a Cloudflare alagúton keresztül éri el a dashboard, ahol WebRTC formátumban kerülnek megjelenítésre.



4.2. ábra. MediaMTX stream pipeline

4.2.2. Képkockák kinyerése

A képkockák kinyerésért a `frame-extractor` szolgáltatás felelős, melynek belépési pontja a `main.py` Python script. Feladata, hogy a három RTSP streamből folyamatosan képkockákat olvasson ki, azokat feldolgozza, majd JSON üzenetként publikálja a `frames-in` Pub/Sub csatornára, ahonnan a `dispatcher` Cloud Függvény továbbítja őket a Vertex AI végpontra. A szolgáltatás a pi-n systemd szolgáltatásként fut, és az `sa-frame-extractor` service account kulcsával hitelesíti magát a Google Cloud felé.

Szálkezelés és capture loop

A script induláskor beolvassa a kamerák RTSP URL-jeit és azonosítóit a környezeti változókból, majd minden egyes kamerához egy dedikált szálat (`threading.Thread`) indít. Minden szál egy önálló `capture_loop` függvényt futtat, amely az OpenCV[18] `VideoCapture` osztályán keresztül csatlakozik az adott RTSP streamhez. Ha a kapcsolat megszakad — például a kamera újraindul vagy a hálózat

elvész —, a loop automatikusan újracsatlakozást kísérel meg 5 másodpercenként, így a rendszer emberi beavatkozás nélkül helyreáll.

Képfeldolgozás és publikálás

Sikeres kapcsolat esetén a loop `CAPTURE_FPS` (alapértelmezetten 2 fps) sebességgel olvas képkockákat. Minden egyes képkockán a következő feldolgozási lépések hajtódnak végre:

1. Az OpenCV a kamera natív felbontásáról átméretezi a képet a konfigurált `FRAME_WIDTH` × `FRAME_HEIGHT` méretre (alapértelmezetten 640×480 pixel).
2. A képkocka JPEG formátumba kerül kódolásra a beállított `JPEG_QUALITY` értékkel (alapértelmezetten 70), csökkentve az átviteli méretet.
3. A JPEG bájttömb Base64 kódolással ASCII szöveggé alakul, hogy JSON üzenetbe ágyazható legyen.
4. A script egy JSON objektumot állít össze, amely tartalmazza a `camera_id` azonosítót, a `seq` szekvenciaszámot, az `ts` UTC-időbélyeget, valamint a `data_b64` mezőt a Base64-kódolt képpel.
5. A kész JSON payload publikálásra kerül a Pub/Sub `frames-in` topicra.

Health check szerver

Mivel a szolgáltatás Cloud Run-kompatibilis konténerként is üzemeltethető, a script egy egyszerű HTTP szervert indít a `PORT` környezeti változóban megadott porton (alapértelmezetten 8080). Ez a szerver minden `GET` kérésre `200 OK` választ ad, lehetővé téve a platform számára, hogy ellenőrizze a konténer élettartamát.

Konfiguráció

A script teljes mértékben környezeti változókon keresztül konfigurálható, nem tartalmaz beégetett értékeket. A legfontosabb paramétereket az alábbi táblázat foglalja össze:

Változó	Alapértelmezett	Leírás
`GCP_PROJECT`	— (kötelező)	Google Cloud projekt azonosítója
`FRAMES_TOPIC`	— (kötelező)	Pub/Sub topic neve
`RTSP_URLS`	— (kötelező)	Vesszővel elválasztott RTSP URL-ek
`CAMERA_IDS`	cam1,cam2,cam3	Kamera azonosítók
`CAPTURE_FPS`	2	Másodpercenkénti képkockák száma
`JPEG_QUALITY`	70	JPEG tömörítési minőség (0–100)
`FRAME_WIDTH`	640	Képkocka szélessége pixelben
`FRAME_HEIGHT`	480	Képkocka magassága pixelben
`PORT`	8080	Health check HTTP szerver portja

4.1. táblázat. A `frame-extractor` konfigurációs paraméterei

4.2.3. Cloudflare alagút

A Cloudflare Tunnel egy fordított proxy megoldás, amely lehetővé teszi, hogy a Pi-n futó szolgáltatások nyilvánosan elérhetők legyenek anélkül, hogy a routeren portot kellene nyitni, vagy statikus IP-cím szükséges lenne[19]. A `cloudflared` démon a Pi-ről kifelé irányuló titkosított kapcsolatot épít fel a Cloudflare hálózata felé, így a tűzfal és a NAT nem akadályozza az elérést. Ez különösen fontos a RTSP videóstreamek dashboardra való eljuttatásakor.

A Pi-n ez is systemd szolgáltatásként fut, az induláskor automatikusan csatlakozik. Jelenleg ideiglenes (*quick tunnel*) alagutakat használunk, amelyek ingyenesek, azonban minden újraindításkor új, véletlenszerű URL-t kapnak, és időnként megszakadhatnak. Egy esetleges éles üzembe helyezés esetén célszerű lenne állandó (*named tunnel*) alagútra váltani, amely fix URL-t biztosít és robusztusabb kapcsolatot nyújt.

4.3. Cloud Stack

A rendszer lelke teljes valójában a google cloudban található, mely igen széleskörű eszköztárat biztosított számomra a projekt megvalósításához. Az infrastruktúra felépítéséhez Terraform keretrendszert

használtunk.

4.3.1. Terraform keretrendszer

A terraform egy deklaratív nyelvezetű keretrendszer, mely használatával bármilyen infrastruktúrai elem és erőforrás definiálható emberek számára is értelmezhető nyelven[20] .Ezen megvalósítások, verziózhatóak, disztributálhatóak és bármikor újrafelhasználhatóak. Segítségével az alkalmazás minden életszakasza könnyűszerrel menedzselhető, ugyanitt maga az egész folyamat rendkivül könnyen automatizálható.

4.3.2. Terraform működése

A terraform a különböző cloud platformok erőforrásait a saját Api-jain keresztül menedzseli, a közösségnek hála ma már szinte bármelyik szolgáltató rendszerét használhatjuk a terraformon keresztül. Például AWS, Google Cloud, Kubernetes, Git stb. A terraform működése 3 fázisban történik:

Írás-Write

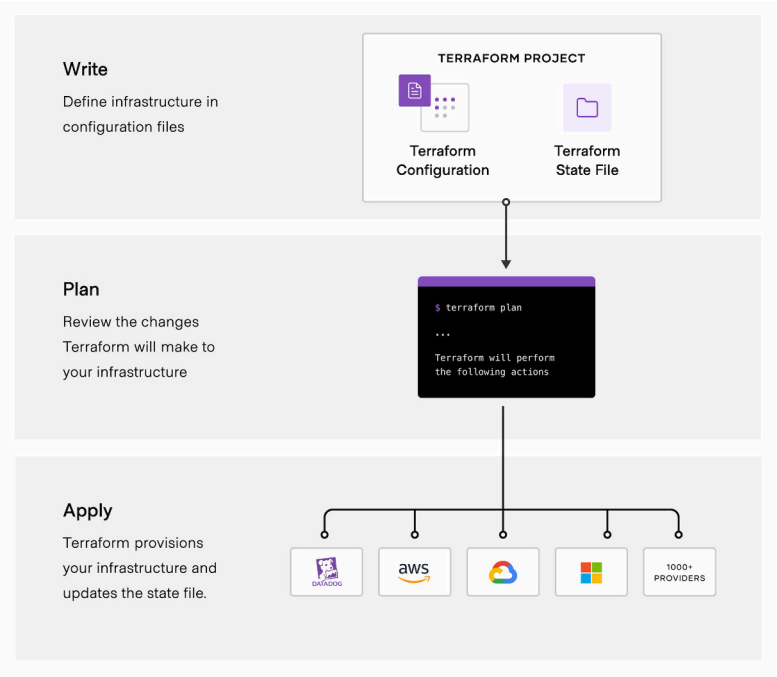
Definiáljuk az erőforrásokat, melyek akár több szolgáltatótól is származhatnak, egy .tf kiterjesztésű fileba.

Tervezés-Plan

A terraform plan parancs kiadása után, létrejön a terv, mely leírja az infrastruktúrát, mi jön létre, mi változik vagy éppen mi szűnik majd meg. Az általunk megadott Konfigurációs fileoknak megfelelően.

Alkalmazás-Apply

Engedélyezés esetén a terraform elvégzi a szükséges műveleteket, tiszteletben tartva az erőforrások közötti függőségi sorrendet. Létrehozza, módosítja vagy törli az érintett erőforrásokat, majd frissíti az állapotfájlt (`terraform.tfstate`), amely a valós infrastruktúra leképezéseként szolgál a jövőbeli futtatásokhoz.



4.3. ábra. Terraform Lépések[1]

4.3.3. Google Cloud Projekt - GCP

A rendszer cloud-oldali erőforrásai egyetlen Google Cloud Projektbe szervezett infrastruktúrán futnak, amelynek egyedi azonosítója `printermonitor-488112`. Az összes felhőszolgáltatás — az Artifact Registry-től a Vertex AI végpontig, a Cloud Function-öktől a Firestore adatbázisig — ezen a projekten belül van definiálva és menedzselve. A Google Cloud Platform (GCP) választásának fő indoka az volt, hogy egységes ökoszisztémát biztosít a gépi tanulási pipeline-hoz: a Vertex AI natívan integrálódik a Pub/Sub üzenetsorral, a Cloud Functions-szel és az Artifact Registry-vel, minimalizálva az integrációs komplexitást[21]. Az erőforrások számlázása és a költségvetési riasztások szintén ezen a projekten belül vannak konfigurálva, lehetővé téve a kiadások szoros nyomon követését.

[1] *Terraform*: https://developer.hashicorp.com/terraform/intro/

4.3.4. Vertex AI végpont

A Vertex AI végpont a detekciós pipeline magja: egy egyedi `judge` Docker konténert futtat, amelyen belül a betanított YOLOv8[14] modell végzi a képkockák valós idejű elemzését. A konténer induláskor betölti a modellt a `MODEL_PATH` által megjelölt `best.pt` fájlból, majd egy HTTP szerveren (`/predict` végpont) vár a bejövő kérésekre.

A szerver kétféle üzenetformátumot kezel: a Vertex AI natív `{"instances": [...]}` formátumát, amelyet a Dispatcher Cloud Function küld, valamint a Pub/Sub push formátumát. Minden beérkező képkockát Base64-ból visszaalakít, OpenCV-vel dekódol, majd a YOLO modellel elvégzi az objektumdetekciót. Az eredmény tartalmazza az észlelt hibaosztályokat (`label`), azok konfidenciaértékeit, valamint a határoló téglalapok koordinátáit mind pixel-, mind normalizált formában. A detekciós eredmény ezt követően publikálásra kerül a `detections-out` Pub/Sub csatornára.

A végpont csak aktív használat közben generál költséget: tétlen (kitelepített, de nem hívott) állapotban nem számlázódik, míg folyamatos terhelés esetén napi körülbelül 37$-os költséggel kell számolni.

4.3.5. Service Accounts

A szervíz felhasználók, olyan egyedi emailcímmel rendelkező entitások, melyek segítségével, különböző applikációk, virtuális gépek, vagy egyéb megvalósítások kommunikálhatnak a Google Api-val, emberi felhasználók helyett, ezzel biztonságos összeköttetést biztosítva a különböző erőforrások között.

sa-frame-extractor

A pi-n fut szerepe, hogy a kinyert képkockák publikálva legyenek a pub/sub csatornára. Jogosultságai, hogy publikál a frames-in csatornára, és aiplatform felhasználó

sa-dispatcher

Az ő szerepete hogy leossza a streameket, és meghívja a VertexAi endpointot. Jogosultságai, hogy aiplatform felhasználó, pub/sub feliratkozó a detections-out csatornán, és aktiválhatja az Alert-managert

szügség esetén

sa-alert-manager

A figyelmeztetés kezelésért felelő, egyaránt az anomáliák észlelésekor, és a költségvetés meghaladásakor. Jogosultságai: `datastore.user`, `firebase.admin`, `eventarc.eventReceiver`, `pubsub.subscriber`, `run.invoker`.

sa-cicd

Mivel az, hogy a terraform infrát mi kézzel szerkesszük nem egy jó praktika, így ehhez egy automatizált github akciót használunk, ezzel szervíz pont ennek kezeléséért felelős. Jogosultságai: `artifactregistry.writer`, `pubsub.admin`, `editor`, `iam.serviceAccountUser`

Vertex AI agent

A Vertex AI endpoint a `judge` (bíró) Docker konténer futtatásáért felelős. Jogosultságai: `pubsub.publisher` a `detections-out` csatornán.

Pub/Sub agent

A Pub/Sub csatornák menedzselésére szolgál. Jogosultságai: `iam.serviceAccountTokenCreator`.

4.3.6. Pub/Sub Topics

A Google Cloud Pub/Sub egy aszinkron üzenetközvetítő szolgáltatás, amely lehetővé teszi az alkalmazáskomponensek közötti laza csatolású kommunikációt[22]. A rendszer három darab pub/sub csatornát használ, melyek a következők: `frames-in`, mely a még nyers képkockákat továbbítja a pi-től a Vertex AI végpont felé. A `detections-out`, mely a Vertex AI végpont által észlelt anomáliák eredményét továbbítja a dashboard felé, és a `budget-notifications`, mely figyelmeztet minket amennyiben átléptük a kiszabott költségvetést.

4.3.7. Google Cloud Függvények

A rendszer három Cloud Function (Gen2) példányt alkalmaz, amelyek Pub/Sub eseményekre triggerelődnek és kiszolgáltatás nélkül, skálázhatóan futnak.Mindhárom függvény az `sa-alert-manager` illetve `sa-dispatcher` service account jogosultságaival rendelkezik.

Dispatcher

A Dispatcher hidat képez az edge eszköz és a Vertex AI végpont között. A `frames-in` Pub/Sub csatornáról kap üzenetet, amelynek tartalma egy Base64-kódolt JPEG képkocka, a kamera azonosítója, egy szekvenciaszám és egy UTC időbélyeg. A függvény ezt az adatot a Vertex AI REST API-jára továbbítja (`/predict` végpont), OAuth2 Bearer tokennel hitelesítve. Ha a Vertex AI kérés hálózati hiba miatt meghiúsul, a függvény újra dobja a kivételt, így a Pub/Sub automatikusan újrapróbálkozik. A három Cloud Függvény közül ez a leginkább memóriaigényes, mivel képkockák bináris adatát szállítja HTTP-n keresztül.

Alert-Manager

A detekciós pipeline záró állomása. A `detections-out` csatornáról érkező üzenetekből kiszűri azokat a detektált hibákat, amelyek konfidenciaértéke eléri a beállított küszöböt (alapértelmezetten 35%). Az átmenő detektálásokat a Firestore `alerts` kollekcióba írja, rögzítve a kamera azonosítóját, az észlelt hibákat, az időbélyeget és a szekvenciaszámot.

E-mail értesítést kizárólag magas súlyosságú hibaosztályok esetén küld: `spagetti`, `not_sticking`, `layer_shift` és `warping`. Az ismétlődő riasztások elkerülése érdekében kameránként 60 másodperces visszatartási időt (*cooldown*) alkalmaz, amelyet a Firestore `alert_cooldowns` kollekcióban tárol. Az e-mail HTML formátumú, tartalmazza az észlelt hibák táblázatát és egy közvetlen linket a dashboardra.

Budget Notifier

E függvény célja, hogy megakadályozza a felhőköltségek elszabadulását. A `budget-notifications` Pub/Sub csatornán hallgatózik, amelyre a GCP saját költségvetési riasztó rendszere publikál, ha az

előre beállított kiadási küszöb teljesül. A függvény az Alert-Managerrel közös kódbázisban van megvalósítva, és szintén cooldown mechanizmust alkalmaz. Az eseményt a Firestore `budget_alerts` kollekcióba is naplózza, majd HTML e-mailben értesíti a felhasználót a felhasznált összegről és a küszöbértékről.

4.3.8. Firestore Collections

Az adatbáziskezelő rendszereknek két nagy fajtáját különböztetünk meg, a relációs adatbázisokat, és a nem relációs avagy NoSQL (Not only SQL) adatbázisokat. A fő különbség , hogy a relácíiós adatbázisokban az adatok táblákba rendezettek, és köztük jól meghatározott kapcsolatok vannak. Viszont a NoSql adatbázisokban az adatok úgynevezett dokumentumokban vannak eltárolva, melyek szerkezete sokkal rugalmasabb és nem igényel előre meghatározott sémát. Mivel a projekt főként json formátumú adatokkal dolgozik, így a NoSQL adatbázisok sokkal jobban megfelelnek a céljainknak, és a Firestore egy igen népszerű és megbízható választás ezen a téren[23].

A rendszer összesen három Firestore kollekciót használ:

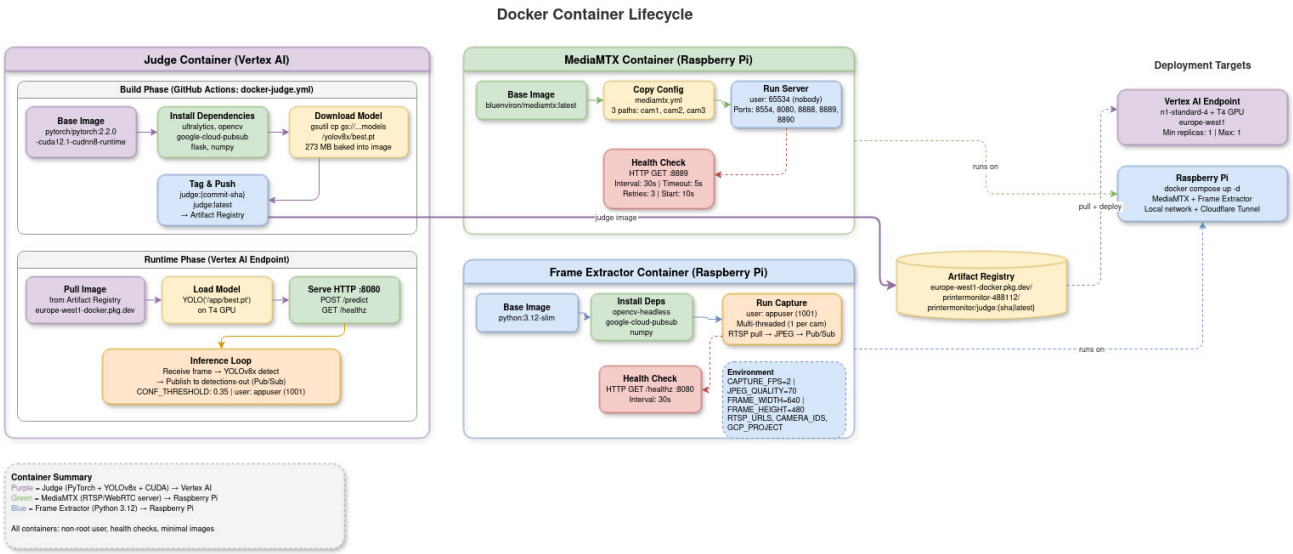
- Az `alerts` kollekció tárolja a detektált anomáliák bejegyzéseit. Mezői: `camera_id` (melyik kamera észlelte), `detections[]` (a hibák tömbje: név + konfidencia + bounding box), `timestamp` (mikor történt a detekció), `seq` (a képkocka szekvenciaszáma, amelyet a `frame-extractor` biztosít) és `created_at` (mikor került be a bejegyzés). A dashboardon való hatékony keresést egy kompozit index segíti: `camera_id ASC` + `timestamp DESC`, azaz kamera szerint növekvő, időbélyeg szerint csökkenő sorrendben.

- Az `alert_cooldowns` kollekció az e-mail értesítések visszatartási idejét kezeli kameránként, megakadályozva a riasztás-özönt rövid időn belül ismétlődő anomáliák esetén.

- A `budget_alerts` kollekció a GCP költségvetési riasztásokat naplózza, beleértve a felhasznált összeget, a beállított küszöböt és az esemény időbélyegét.

4.3.9. Konténer képek

A Vertex AI végponton futó detekciós logika egy egyedi Docker konténer képbe (image) van csomagolva, melynek neve `judge`. Ez az image egy Python scriptet tartalmaz, amely induláskor betölti a betanított YOLOv8 modellt, majd várakozik a bejövő Pub/Sub üzenetekre. Egy üzenet érkezésekor kicsomagolja a base64-kódolt JPEG képkockát, elvégzi rajta az objektumdetekciót, majd az eredményt — a detektált hibaosztályokat és azok konfidenciaértékeit — visszapublikálja a `detections-out` Pub/Sub csatornára[24]. Az image tárolása a Google Artifact Registry-ben történik, verziózva a commit SHA alapján. Minden módosítás esetén a `docker-judge.yml` GitHub Action automatikusan újrabuildi és feltölti a friss image-t, ahonnan a Vertex AI végpont azt lehúzza és futtatja.



4.4. ábra. A `judge` konténer image életciklusa

4.4. Dashboard

A rendszer felhasználói felülete egy egyoldalas webalkalmazás (Single Page Application), melyet a Firebase Hosting platform szolgál ki, így külön szerver üzemeltetése nélkül érhető el a böngészőből[25]. Az alkalmazás két fő funkciót lát el párhuzamosan.

Egyrészt valós időben jeleníti meg a három kamera videofolyamát WebRTC technológia segítségével, a MediaMTX szerveren és a Cloudflare alagúton keresztül érkező RTSP streamekre támasz-

kodva. Másrészt a Firebase Firestore adatbázishoz való közvetlen, valós idejű kapcsolaton keresztül (`onSnapshot` listener) azonnal megjelenik minden újonnan detektált anomália, azok neve és konfidenciaértéke, valamint az észlelés időbélyege és a forrás kamera azonosítója.

A dashboard vizuálisan is visszajelzést ad a rendszer állapotáról: amennyiben magas prioritású anomália érkezik, a felület piros kerettel jelenik meg (CSS `alerting` osztály aktiválásával). Hasonlóan, a költségvetési riasztás esetén sárga kerettel jelez (`budget-alert` osztály), így a felhasználó azonnal értesül a túllépésről anélkül, hogy az e-mailjét kellene ellenőriznie.

[ÁBRA: Dashboard normál nézet screenshot]

4.5. ábra. A dashboard normál működési állapotban

[ÁBRA: Dashboard riasztási állapot screenshot (piros keret)]

4.6. ábra. A dashboard anomália detektálás esetén

4.5. Github Action Workflows

A projekt folyamatos integrációját és automatizált kitelepítését (CI/CD) két GitHub Action workflow biztosítja, melyek a megfelelő kódváltozásra automatikusan lefutnak, csökkentve a manuális beavatkozás szükségességét és az emberi hibák esélyét[26].

4.5.1. Terraform lépések automatizálása

Ez a workflow a cloud infrastruktúra változtatásait kezeli automatikusan. Pull request esetén csak ellenőrző lépések futnak le, hogy a változtatások biztonságosak és helyesek legyenek merge előtt. Merge eseményre — a `main` ágra — a workflow elvégzi a tényleges infrastruktúra módosítást a GCP-n.

Konfigurációs ellenőrzés-Chekov

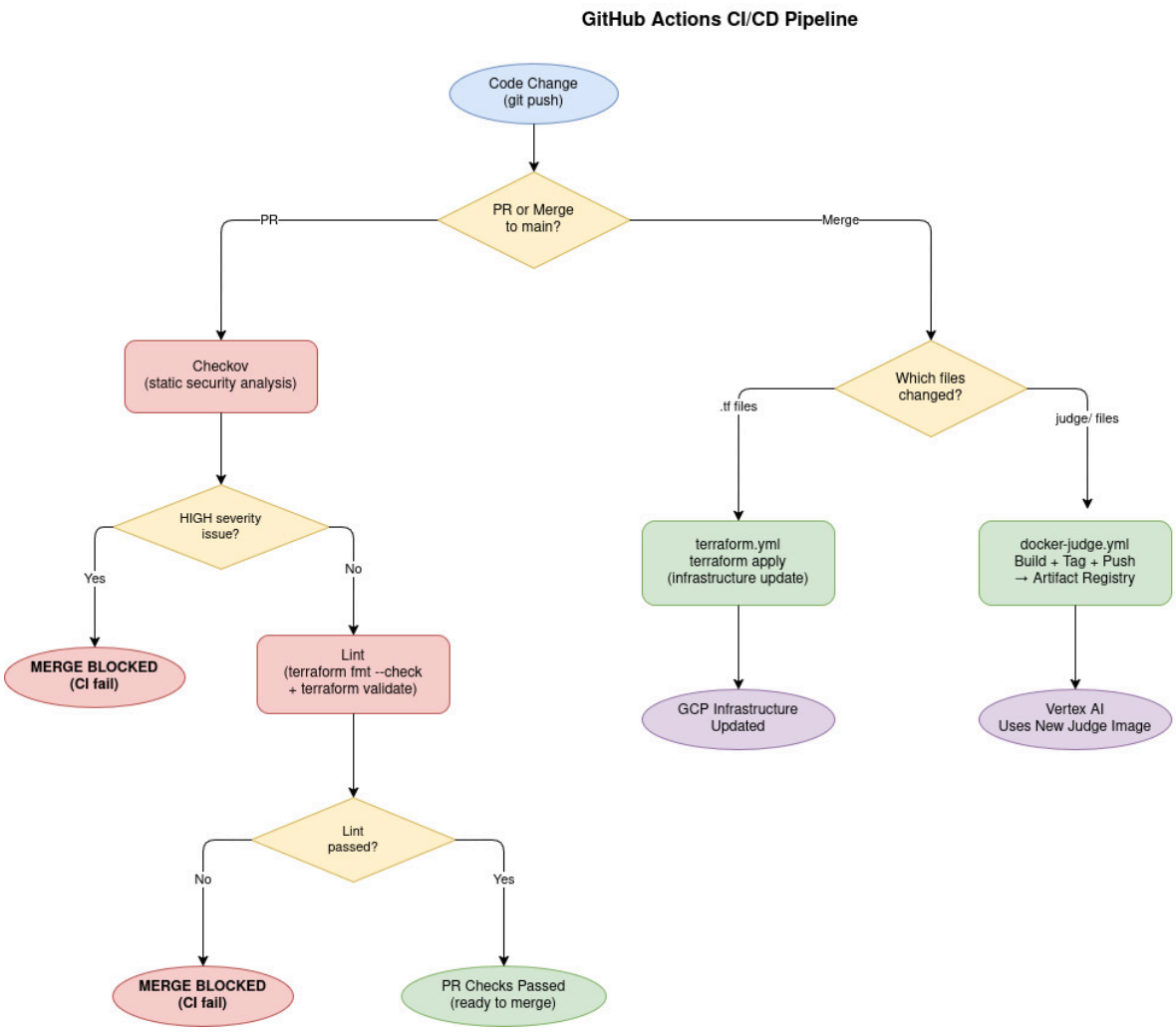
A Checkov egy nyílt forráskódú statikus elemző eszköz, amely Terraform konfigurációkat vizsgál ismert biztonsági problémák és rossz praktikák szempontjából[27]. Futtatása pull request esetén kötelező lépés: amennyiben magas súlyosságú problémát talál — például nyilvánosan elérhető storage bucket vagy nem megfelelő IAM konfiguráció —, a workflow megakadályozza a merge-t. Ez biztosítja, hogy a fejlesztés során se kerülhessen sérülékeny infrastruktúra a produkcióba.

Formázás és erőforráshasználat ellenőrzés-Lint

A linting lépés két dolgot ellenőriz. A `terraform fmt -check` parancs megvizsgálja, hogy a `.tf` fájlok formázása megfelel-e a Terraform hivatalos stílusirányelveknek. A `terraform validate` parancs pedig szintaktikailag és szemantikailag ellenőrzi a konfigurációt, kiszűrve a nem létező erőforrásokra való hivatkozásokat vagy érvénytelen argumentumokat. Ezzel a lépéssel a triviális hibák már a review fázisban kiszűrhetők.

4.5.2. Docker konténer felépítésének automatizálása

Ez a workflow a `judge` Docker image buildjeléért és az Artifact Registry-be való automatikus feltöltéséért felelős. Minden olyan push eseményre lefut, amely a `judge/` könyvtárat érinti — így csak releváns változtatás esetén indul el, nem feleslegesen. A workflow lépései sorban: hitelesítés az `sa-cicd` service account kulcsával, az image buildje, ellátása a commit SHA-val mint egyedi taggel, majd feltöltése a Google Artifact Registry-be. Ezt követően a Vertex AI végpont a következő aktiváláskor automatikusan az új image-t fogja használni.



4.7. ábra. A CI/CD pipeline lépései

Összefoglalás

A fejezet bemutatta a rendszer teljes hardver- és szoftverkörnyezetét. Az edge oldalon egy Raspberry Pi 4B gondoskodik a kameraképek rögzítéséről és valós idejű továbbításáról, a felhő oldalon pedig a Google Cloud Platform szolgáltatásai — Pub/Sub, Cloud Functions, Vertex AI, Firestore és Firebase Hosting — alkotják azt a pipeline-t, amely a nyers képkockáktól az anomáliadetekción át a felhasználói értesítésig minden lépést lefed. Az infrastruktúra Terraformmal van definiálva és verziókezelve, a CI/CD pipeline pedig biztosítja, hogy minden kódváltozás automatikusan, emberi beavatkozás nélkül

jusson el az éles környezetbe.

5. fejezet

Adatgyűjtés és előfeldolgozás

5.1. Cím 1

6. fejezet

Modell betanítása és validálása

6.1. Cím 1

Eredmények leírása

7. fejezet

Tesztelés és kiértékelés

7.1. Összefoglalás

8. fejezet

Rendszerintegráció és alkalmazás

9. fejezet

Összefoglalás és jövőbeli irányok

10. fejezet

Mellékletek

Irodalomjegyzék

[1] Voxel Matters, „The Rise of 3D Printer Farms.” https://www.voxelmatters.com/the-rise-of-3d-printer-farms/, 2023. Hozzáférés: 2026.

[2] Obico, „Obico: Open-Source AI-Powered 3D Printing Failure Detection.” https://www.obico.io/, 2021. Hozzáférés: 2026.

[3] All3DP, „3D Printer Filament Types – The Definitive Guide.” https://all3dp.com/1/3d-printer-filament-types-3d-printing-3d-filament/, 2024. Hozzáférés: 2026.

[4] Bambu Lab, „Bambu Lab Wiki – 3D Printing Knowledge Base.” https://wiki.bambulab.com/, 2024. Hozzáférés: 2026.

[5] Bambu Lab, „Stringing and Oozing – Bambu Lab Wiki.” https://wiki.bambulab.com/en/filament-acc/filament/print-quality/stringing-oozing, 2024. Hozzáférés: 2026.

[6] Bambu Lab, „Printed Model Warping – Bambu Lab Wiki.” https://wiki.bambulab.com/en/knowledge-sharing/printed-model-warping, 2024. Hozzáférés: 2026.

[7] Bambu Lab, „Layer Shifts – Bambu Lab Wiki.” https://wiki.bambulab.com/en/knowledge-sharing/layer-shifts, 2024. Hozzáférés: 2026.

[8] Bambu Lab, „Under Extrusion – Bambu Lab Wiki.” https://wiki.bambulab.com/en/filament-acc/filament/print-quality/under-extrusion, 2024. Hozzáférés: 2026.

[9] Bambu Lab, „Nozzle Clog – Bambu Lab Wiki.” https://wiki.bambulab.com/en/filament-acc/filament/print-quality/clog, 2024. Hozzáférés: 2026.

[10] Bambu Lab, „First Layer Not Sticking – Bambu Lab Wiki." https://wiki.bambulab.com/en/knowledge-sharing/first-layer-not-sticking, 2024. Hozzáférés: 2026.

[11] Bambu Lab, „Spaghetti Detection – Bambu Lab Wiki." https://wiki.bambulab.com/en/knowledge-sharing/Spaghetti_detection, 2024. Hozzáférés: 2026.

[12] S. Ren, K. He, R. Girshick, and J. Sun, „Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[13] G. Jocher, „YOLOv5 by Ultralytics." https://github.com/ultralytics/yolov5, 2020. Hozzáférés: 2026.

[14] Ultralytics, „YOLOv8: A new state-of-the-art computer vision model." https://github.com/ultralytics/ultralytics, 2023. Hozzáférés: 2026.

[15] Intel Corporation, „CVAT: Computer Vision Annotation Tool." https://github.com/cvat-ai/cvat, 2019. Hozzáférés: 2026.

[16] Raspberry Pi Ltd., „Raspberry Pi 4 Model B." https://www.raspberrypi.com/products/raspberry-pi-4-model-b/, 2024. Hozzáférés: 2026.

[17] bluenviron, „MediaMTX: Ready-to-use SRT / WebRTC / RTSP / RTMP / LL-HLS media server." https://github.com/bluenviron/mediamtx, 2024. Hozzáférés: 2026.

[18] OpenCV Team, „OpenCV: Open Source Computer Vision Library." https://opencv.org/, 2024. Hozzáférés: 2026.

[19] Cloudflare Inc., „Cloudflare Tunnel Documentation." https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, 2024. Hozzáférés: 2026.

[20] HashiCorp Inc., „Terraform: Infrastructure as Code." https://developer.hashicorp.com/terraform/intro, 2024. Hozzáférés: 2026.

[21] Google LLC, „Google Cloud Platform Documentation." https://cloud.google.com/docs, 2024. Hozzáférés: 2026.

[22] Google LLC, „Google Cloud Pub/Sub Documentation.” https://cloud.google.com/pubsub/docs, 2024. Hozzáférés: 2026.

[23] Google LLC, „Cloud Firestore Documentation.” https://firebase.google.com/docs/firestore, 2024. Hozzáférés: 2026.

[24] Google LLC, „Vertex AI: Use a custom container for prediction.” https://cloud.google.com/vertex-ai/docs/predictions/use-custom-container, 2024. Hozzáférés: 2026.

[25] Google LLC, „Firebase Hosting Documentation.” https://firebase.google.com/docs/hosting, 2024. Hozzáférés: 2026.

[26] GitHub Inc., „GitHub Actions Documentation.” https://docs.github.com/en/actions, 2024. Hozzáférés: 2026.

[27] Bridgecrew Inc., „Checkov: Static code analysis tool for infrastructure as code.” https://www.checkov.io/, 2024. Hozzáférés: 2026.

A. függelék

Függelék

A.1. Alfejezet

A.1.1. Cím

Alcím